

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part – 1

```
CREATE TABLE customers (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  signup_date DATE NOT NULL  
);
```

```
CREATE TABLE restaurants (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  cuisine VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  restaurant_id INT NOT NULL,  
  total DECIMAL(10, 2) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  created_at DATETIME NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(id),  
  FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)  
);
```

```
CREATE TABLE accounts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  balance DECIMAL(10, 2) NOT NULL
```

);

```
INSERT INTO customers (name, email, signup_date) VALUES
('Alice Smith', 'alice@example.com', '2026-01-10'),
('Bob Jones', 'bob@example.com', '2026-01-12'),
('Charlie Brown', 'charlie@example.com', '2026-01-15'),
('David Miller', 'david@example.com', '2026-01-20'),
('Emma Wilson', 'emma@example.com', '2026-02-01'),
('Frank Thomas', 'frank@example.com', '2026-02-03'),
('Grace Davis', 'grace@example.com', '2026-02-10'),
('Henry Garcia', 'henry@example.com', '2026-02-14'),
('Ivy Martinez', 'ivy@example.com', '2026-02-20'),
('Jack Robinson', 'jack@example.com', '2026-03-01'),
('James Clark', 'james@example.com', '2026-03-05'),
('Kendra Lewis', 'kendra@example.com', '2026-03-12'),
('Liam Walker', 'liam@example.com', '2026-03-25'),
('Mia Hall', 'mia@example.com', '2026-04-01'),
('Noah Allen', 'noah@example.com', '2026-04-05');
```

```
INSERT INTO restaurants (name, cuisine) VALUES
('Spice Symphony', 'Indian'),
('Pasta Palace', 'Italian'),
('Dragon Wok', 'Chinese'),
('Burger Bistro', 'American'),
('Taco Fiesta', 'Mexican'),
('Sushi Sakura', 'Japanese'),
('Le Gourmet', 'French'),
('Olive Grove', 'Mediterranean');
```

```
INSERT INTO orders (customer_id, restaurant_id, total, status, created_at) VALUES
(1, 1, 450.00, 'completed', '2026-01-11 12:30:00'),
(1, 2, 850.50, 'completed', '2026-01-15 19:15:00'),
(2, 3, 320.00, 'completed', '2026-01-13 13:00:00'),
(2, 4, 600.00, 'cancelled', '2026-01-20 20:00:00'),
```

(3, 1, 1200.00, 'completed', '2026-01-16 21:00:00'),
(3, 5, 250.00, 'completed', '2026-01-22 11:45:00'),
(4, 6, 1500.00, 'completed', '2026-01-22 18:30:00'),
(4, 2, 720.00, 'pending', '2026-01-25 19:00:00'),
(5, 7, 2400.00, 'completed', '2026-02-02 20:30:00'),
(5, 3, 350.00, 'completed', '2026-02-05 13:10:00'),
(6, 4, 480.00, 'completed', '2026-02-04 12:00:00'),
(6, 8, 950.00, 'completed', '2026-02-10 14:20:00'),
(7, 1, 550.00, 'completed', '2026-02-12 19:45:00'),
(7, 5, 310.00, 'completed', '2026-02-18 12:15:00'),
(8, 2, 890.00, 'completed', '2026-02-15 20:00:00'),
(8, 6, 1350.00, 'completed', '2026-02-22 18:00:00'),
(9, 3, 420.00, 'completed', '2026-02-21 13:30:00'),
(9, 7, 1850.00, 'pending', '2026-02-27 21:15:00'),
(10, 4, 650.00, 'completed', '2026-03-02 12:45:00'),
(10, 8, 780.00, 'completed', '2026-03-08 19:00:00'),
(11, 1, 380.00, 'completed', '2026-03-06 13:00:00'),
(11, 2, 920.00, 'completed', '2026-03-10 20:30:00'),
(12, 5, 290.00, 'completed', '2026-03-14 11:30:00'),
(12, 6, 1600.00, 'completed', '2026-03-20 18:45:00'),
(1, 3, 410.00, 'completed', '2026-02-01 12:00:00'),
(2, 7, 2100.00, 'completed', '2026-02-15 20:15:00'),
(3, 4, 550.00, 'completed', '2026-02-20 13:00:00'),
(4, 8, 880.00, 'completed', '2026-03-01 19:30:00'),
(5, 1, 620.00, 'completed', '2026-03-05 12:30:00'),
(6, 2, 790.00, 'completed', '2026-03-12 20:00:00'),
(7, 3, 490.00, 'completed', '2026-03-20 13:15:00'),
(8, 4, 610.00, 'completed', '2026-03-25 19:45:00'),
(9, 5, 330.00, 'completed', '2026-03-28 12:00:00'),
(10, 6, 1250.00, 'completed', '2026-04-01 18:30:00'),
(11, 7, 1950.00, 'completed', '2026-04-03 21:00:00'),
(12, 8, 820.00, 'completed', '2026-04-06 14:00:00'),
(1, 4, 530.00, 'completed', '2026-03-15 13:00:00'),
(2, 5, 270.00, 'completed', '2026-04-02 12:15:00'),
(3, 6, 1400.00, 'completed', '2026-04-05 19:00:00'),

(4, 1, 460.00, 'completed', '2026-04-07 12:30:00');

INSERT INTO accounts (name, balance) VALUES
('Alice', 1000.00),
('Bob', 1000.00);

Part – 2

TASK 1 Transactions (ACID)

a)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 START TRANSACTION;  
2  
3 UPDATE accounts  
4 SET balance = balance - 200.00  
5 WHERE name = 'Alice';  
6  
7 UPDATE accounts  
8 SET balance = balance + 200.00  
9 WHERE name = 'Bob';  
10  
11 ROLLBACK;  
12  
13 SELECT * FROM accounts;
```

Results

Copy as Markdown

Query #1 Execution time: 0.00ms

There are no results to be displayed.

Query #5 Execution time: 0.14ms

id	name	balance
1	Alice	1000.0
2	Bob	1000.0

b)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 START TRANSACTION;  
2  
3 UPDATE accounts  
4 SET balance = balance - 200.00  
5 WHERE name = 'Alice';  
6  
7 UPDATE accounts  
8 SET balance = balance + 200.00  
9 WHERE name = 'Bob';  
10  
11 COMMIT;  
12  
13 SELECT * FROM accounts;
```

Results

Copy as Markdown

Query #1 Execution time: 0.00ms

There are no results to be displayed.

Query #5 Execution time: 0.16ms

id	name	balance
1	Alice	800.0
2	Bob	1200.0

c)

The ACID property that guarantees money is never "lost" in the middle of a transfer is Atomicity. Atomicity ensures that a transaction is treated as a single, indivisible unit of work—meaning either all statements succeed completely, or the entire transaction fails and leaves the database completely unchanged. This prevent scenarios where money is successfully debited from one account but never credited to the target account due to a system crash midway.

TASK 2 Joins

a)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 SELECT  
2   c.id AS customer_id,  
3   c.name AS customer_name,  
4   c.email,  
5   o.id AS order_id,  
6   o.total AS order_total  
7 FROM customers c  
8 INNER JOIN orders o ON c.id = o.customer_id;
```

Results

Copy as Markdown

Query #1 Execution time: 0.45ms

customer_id	customer_name	email	order_id	order_total
1	Alice Smith	alice@example.com	1	450.0
1	Alice Smith	alice@example.com	2	850.5
1	Alice Smith	alice@example.com	25	410.0
1	Alice Smith	alice@example.com	37	530.0

b)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 SELECT  
2   c.id AS customer_id,  
3   c.name AS customer_name,  
4   c.email,  
5   c.signup_date  
6 FROM customers c  
7 LEFT JOIN orders o ON c.id = o.customer_id  
8 WHERE o.customer_id IS NULL;
```

Results

Copy as Markdown

Query #1 Execution time: 0.3ms

customer_id	customer_name	email	signup_date
13	Liam Walker	liam@example.com	2026-03-25
14	Mia Hall	mia@example.com	2026-04-01
15	Noah Allen	noah@example.com	2026-04-05

c)

A marketing team would highly prefer the result of query (b) because it explicitly isolates the company's inactive users (customers who signed up but have never placed an order), allowing the team to target them directly with re-engagement email campaigns, promotional discounts, or first-order incentives.

TASK 3 Indexing

a)

We will use the **status** column from the orders table, which currently does not have an index.

b)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Query SQL

```
1 EXPLAIN ANALYZE  
2 SELECT * FROM orders  
3 WHERE status = 'pending';
```

Results

Query #1 Execution time: 0.38ms

EXPLAIN

-> Filter: (orders.'status' = 'pending') (cost=4.25 rows=4) (actual time=0.0298..0.0372 rows=2 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0201..0.0266 rows=40 loops=1)

c) & d)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Query SQL

```
1 CREATE INDEX idx_orders_status ON orders(status);  
2  
3 EXPLAIN ANALYZE  
4 SELECT * FROM orders  
5 WHERE status = 'pending';
```

Results

Query #1 Execution time: 18.97ms

There are no results to be displayed.

Query #2 Execution time: 1.16ms

EXPLAIN

-> Index lookup on orders using idx_orders_status (status = 'pending') (cost=0.7 rows=2) (actual time=0.0147..0.016 rows=2 loops=1)

e) Before adding the index, MySQL had to perform a full table scan examining all 40 rows; after indexing, it performed an instant index lookup examining only the 2 matching rows, significantly cutting down execution time.

TASK 4 Query Optimization

Base query:

```
SELECT * FROM orders WHERE customer_id IN (  
    SELECT id FROM customers WHERE signup_date >= '2026-01-15'  
);  
a)
```

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 SELECT orders.* FROM orders  
2 INNER JOIN customers ON orders.customer_id = customers.id  
3 WHERE customers.signup_date >= '2026-01-15';
```

Results

Copy as Markdown

Query #1 Execution time: 0.44ms

id	customer_id	restaurant_id	total	status	created_at
5	3	1	1200.0	completed	2026-01-16 21:00:00
6	3	5	250.0	completed	2026-01-22 11:45:00
27	3	4	550.0	completed	2026-02-20 13:00:00
39	3	6	1400.0	completed	2026-04-05 19:00:00

b)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 SELECT  
2   orders.id AS order_id,  
3   orders.total,  
4   orders.status,  
5   orders.created_at,  
6   customers.name AS customer_name  
7 FROM orders  
8 INNER JOIN customers ON orders.customer_id = customers.id  
9 WHERE customers.signup_date >= '2026-01-15';
```

Results

Copy as Markdown

Query #1 Execution time: 0.36ms

order_id	total	status	created_at	customer_name
5	1200.0	completed	2026-01-16 21:00:00	Charlie Brown
6	250.0	completed	2026-01-22 11:45:00	Charlie Brown
27	550.0	completed	2026-02-20 13:00:00	Charlie Brown
39	1400.0	completed	2026-04-05 19:00:00	Charlie Brown

c)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13  
14 CREATE TABLE orders (  
15   id INT AUTO_INCREMENT PRIMARY KEY,  
16   customer_id INT NOT NULL,
```

Text to DDL

Query SQL

```
1  
2 EXPLAIN ANALYZE  
3 SELECT * FROM orders WHERE customer_id IN (  
4   SELECT id FROM customers WHERE signup_date >= '2026-01-15'  
5 );  
6  
7 EXPLAIN ANALYZE  
8 SELECT orders.id, orders.total, orders.status, orders.created_at, customers.name  
9 FROM orders  
10 INNER JOIN customers ON orders.customer_id = customers.id  
11 WHERE customers.signup_date >= '2026-01-15';
```

Results

EXPLAIN

-> Nested loop inner join (cost=3.5 rows=5) (actual time=0.0386..0.0854 rows=32 loops=1) -> Filter: (customers.signup_date >= DATE'2026-01-15') (cost=1.75 rows=5) (actual time=0.0169..0.0216 rows=13 loops=1) -> Table scan on customers (cost=1.75 rows=15) (actual time=0.014..0.0184 rows=15 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.27 rows=1) (actual time=0.0038..0.00431 rows=2.46 loops=13)

Copy as Markdown

Query #2

Execution time: 0.54ms

EXPLAIN

-> Nested loop inner join (cost=3.5 rows=5) (actual time=0.0457..0.0935 rows=32 loops=1) -> Filter: (customers.signup_date >= DATE'2026-01-15') (cost=1.75 rows=5) (actual time=0.0257..0.0307 rows=13 loops=1) -> Table scan on customers (cost=1.75 rows=15) (actual time=0.0227..0.0275 rows=15 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.27 rows=1) (actual time=0.0039..0.00442 rows=2.46 loops=13)

TASK 5 The Customer Profile Page

a)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 SELECT orders.*, restaurants.name AS restaurant_name  
2 FROM orders  
3 INNER JOIN restaurants ON orders.restaurant_id = restaurants.id  
4 WHERE orders.customer_id = 1  
5 ORDER BY orders.created_at DESC  
6 LIMIT 5;
```

Results

Copy as Markdown

Query #1

Execution time: 0.37ms

id	customer_id	restaurant_id	total	status	created_at	restaurant_name
37	1	4	530.0	completed	2026-03-15 13:00:00	Burger Bistro
25	1	3	410.0	completed	2026-02-01 12:00:00	Dragon Wok
2	1	2	850.5	completed	2026-01-15 19:15:00	Pasta Palace
1	1	1	450.0	completed	2026-01-11 12:30:00	Spice Symphony

b)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13  
-- -----  
-- -----
```

Text to DDL

Query SQL

```
1 EXPLAIN ANALYZE  
2 SELECT orders.*, restaurants.name AS restaurant_name  
3 FROM orders  
4 INNER JOIN restaurants ON orders.restaurant_id = restaurants.id  
5 WHERE orders.customer_id = 1  
6 ORDER BY orders.created_at DESC  
7 LIMIT 5;
```

Results

Copy as Markdown

Query #1 Execution time: 2.32ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.55 rows=4) (actual time=0.0569..0.0762 rows=4 loops=1) -> Nested loop inner join (cost=2.55 rows=4) (actual time=0.0534..0.0724 rows=4 loops=1) -> Sort: orders.created_at DESC (cost=1.15 rows=4) (actual time=0.0466..0.047 rows=4 loops=1) -> Index lookup on orders using customer_id (customer_id = 1) (cost=1.15 rows=4) (actual time=0.0243..0.0263 rows=4 loops=1) -> Single-row index lookup on restaurants using PRIMARY (id = orders.restaurant_id) (cost=0.275 rows=1) (actual time=0.00226..0.0023 rows=1 loops=4)

c) & d)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13  
-- -----  
-- -----
```

Text to DDL

Query SQL

```
1 EXPLAIN ANALYZE  
2 SELECT orders.*, restaurants.name AS restaurant_name  
3 FROM orders  
4 INNER JOIN restaurants ON orders.restaurant_id = restaurants.id  
5 WHERE orders.customer_id = 1  
6 ORDER BY orders.created_at DESC  
7 LIMIT 5;
```

Results

Copy as Markdown

Query #1 Execution time: 2.32ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.55 rows=4) (actual time=0.0569..0.0762 rows=4 loops=1) -> Nested loop inner join (cost=2.55 rows=4) (actual time=0.0534..0.0724 rows=4 loops=1) -> Sort: orders.created_at DESC (cost=1.15 rows=4) (actual time=0.0466..0.047 rows=4 loops=1) -> Index lookup on orders using customer_id (customer_id = 1) (cost=1.15 rows=4) (actual time=0.0243..0.0263 rows=4 loops=1) -> Single-row index lookup on restaurants using PRIMARY (id = orders.restaurant_id) (cost=0.275 rows=1) (actual time=0.00226..0.0023 rows=1 loops=4)

e)

Schema SQL

```
1 CREATE TABLE customers (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) UNIQUE NOT NULL,  
5   signup_date DATE NOT NULL  
6 );  
7  
8 CREATE TABLE restaurants (  
9   id INT AUTO_INCREMENT PRIMARY KEY,  
10  name VARCHAR(100) NOT NULL,  
11  cuisine VARCHAR(50) NOT NULL  
12 );  
13
```

Text to DDL

Query SQL

```
1 CREATE INDEX idx_orders_customer_date ON orders(customer_id, created_at DESC);  
2  
3 SELECT  
4   restaurants.name AS restaurant_name,  
5   orders.total AS order_total,  
6   orders.status AS order_status,  
7   orders.created_at  
8 FROM orders  
9 INNER JOIN restaurants ON orders.restaurant_id = restaurants.id  
10 WHERE orders.customer_id = 1  
11 ORDER BY orders.created_at DESC  
12 LIMIT 5;
```

Results

Copy as Markdown

Query #2 Execution time: 0.49ms

restaurant_name	order_total	order_status	created_at
Burger Bistro	530.0	completed	2026-03-15 13:00:00
Dragon Wok	410.0	completed	2026-02-01 12:00:00
Pasta Palace	850.5	completed	2026-01-15 19:15:00
Spice Symphony	450.0	completed	2026-01-11 12:30:00

f)

Performance Comparison: Before applying the index, the query executed a full table scan over all 40 rows and manually sorted them in memory. After creating the composite index, the database performed an immediate index lookup, examining only the targeted customer rows while skipping the sorting phase entirely.

Why the Index Helped: The composite index (customer_id, created_at DESC) was highly effective because it directly matches both the filtering logic (WHERE customer_id) and the sorting arrangement (ORDER BY created_at DESC) required by the query. This allowed MySQL to pull the 5 most recent orders sequentially right out of the index storage without having to read unrelated database records or perform an expensive internal sort operation.